

GAN-based Modelling for Operational Risk

Edinburgh Summer School on Generative AI for Extremes

Luca Trapin

Department of Statistical Sciences
University of Bologna

8–11 September 2026
JCMB, Edinburgh



THE UNIVERSITY
of EDINBURGH



Who am I?

- I am **Associate Professor of Economic Statistics** at the Department of Statistical Sciences, University of Bologna.
- My background is in finance, economics and extreme value statistics.
- My research focuses on statistical models for extremes, financial risk management and time series.
- I am also a member of:
 - **GAMEX**: Generative AI Modelling for EXtreme Events;
 - **CIRCE**: Community of Italian Researchers Connecting on Extreme Value Statistics.

From generative AI for extremes to operational risk

- This school is about **generative models** for extreme events.
- In an extremes setting, realistic synthetic data should look plausible not only in the center of the distribution, but also in the **tail**.
- This is especially important in financial risk, where a small number of extreme losses can dominate the risk profile.
- In this talk, we use **operational risk** as a case study: losses caused by failures of processes, people, systems or external events.
 - ⇒ This setting combines **several difficulties** of generative modelling for extremes: limited data, heavy-tailed losses and dependence across event types.

Guiding question

- We use this case study to compare different **GAN-based approaches** for generating synthetic operational loss data.
- The guiding question is: can these methods generate synthetic losses that preserve the risk-relevant features of the data?
- In particular, we will focus on:
 - heavy-tailed marginal behavior of each event type;
 - dependence across event types;
 - aggregate losses and risk measures such as VaR and ES.
- In this setting, a generative model is useful only if it preserves the features that matter for **risk**.

- 1 Financial risk as an extremes problem
- 2 Operational risk and data challenges
- 3 Synthetic operational loss generation
- 4 Tail-aware GAN methods
- 5 Empirical results

Financial risk as an extremes problem

- **Financial risk** is the possibility of suffering losses because future outcomes are uncertain.
- For a **financial institution**, uncertainty can come from markets, borrowers, operations or external events.
- Financial institutions manage risk for two main reasons:
 - from an **internal perspective**, they need to understand vulnerabilities and allocate capital;
 - from a **regulatory perspective**, they must hold enough capital to absorb severe losses.
- In both cases, the central object is a **future loss**.

Main sources of financial losses

- Future losses may come from different **sources**:
 - **Market risk**: losses due to changes in market prices, interest rates, exchange rates or returns.
 - **Credit risk**: losses due to defaults, changes in credit quality, exposures and recovery rates.
 - **Operational risk**: losses due to failures of processes, people, systems or external events.
- This talk focuses on **operational risk**, where losses are generated by events rather than by market prices or borrower defaults.

The loss distribution

- Let L_t be the loss at time t . We observe losses up to time T , i.e. we collect a sample

$$L_1, \dots, L_T$$

- Risk management focuses on the future loss L_{T+h} , where h is the **risk horizon**.
- The mechanism generating L_{T+h} depends on the **risk class**.
 - $\implies$ in market risk, losses are induced by future changes in prices, rates or returns;
 - $\implies$ in credit risk, losses are induced by defaults, exposures and recovery rates;
 - $\implies$ in operational risk, losses are induced by the occurrence and severity of loss events.
- The statistical object is the distribution of L_{T+h} .

- Let L_{T+h} be the future loss, and let $\alpha \in (0, 1)$ be a high confidence level.
- Common risk measures in the literature are:

- **Value-at-Risk** is the loss level exceeded with probability $1 - \alpha$, defined as:

$$\text{VaR}_\alpha(L_{T+h}) = q_\alpha(L_{T+h}).$$

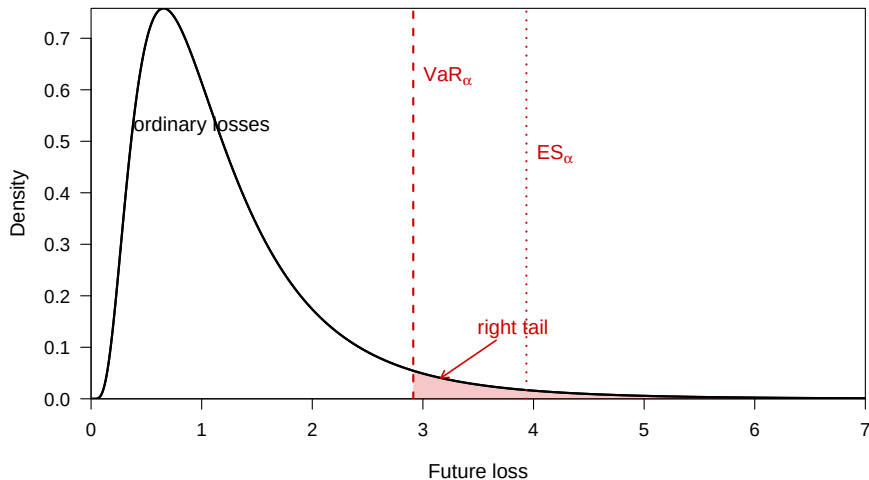
where $q_\alpha(L_{T+h})$ is the quantile of L_{T+h} at level α .

- **Expected Shortfall** is the average loss beyond VaR, defined as:

$$\text{ES}_\alpha(L_{T+h}) = \mathbb{E}[L_{T+h} \mid L_{T+h} > \text{VaR}_\alpha(L_{T+h})].$$

$\implies$ Both quantities focus on **severe losses**.

The loss distribution



The **right tail** of the future loss distribution is the relevant region for extreme-loss analysis.

Operational risk and data challenges

What is operational risk?

- Operational risk is formally defined in the **Basel II framework**.

"The risk of loss resulting from inadequate or failed internal processes, people and systems or from external events. This definition includes legal risk, but excludes strategic risk and reputational risk. "

- Operational risk is not primarily driven by market prices, as in market risk, or by borrower defaults, as in credit risk.
- It is generated by the way a financial institution is organized, operated, controlled and exposed to external events.

Concrete examples of operational loss events

- **Payment fraud.** A client receives a phishing message, credentials are stolen, and fraudulent transfers are executed before the bank detects the anomaly. The loss may include reimbursed funds, investigation costs and control remediation.
- **System outage.** A core payment or trading system becomes unavailable for several hours. Transactions are delayed, clients are affected, manual procedures are activated and the bank may face compensation claims or regulatory scrutiny.
- **Execution or processing error.** A transaction is booked with the wrong amount, wrong counterparty or wrong product code. If the error is detected late, the correction may generate losses, disputes and operational recovery costs.
- **Legal or conduct event.** A product is sold with inadequate disclosure or weak suitability checks. Years later, the bank may face litigation, settlements, fines and compensation payments.

Large operational losses in practice

- **Barings Bank, 1995.** Unauthorized derivatives trading by Nick Leeson was hidden in internal accounts. Losses reached about GBP 827 million and the bank collapsed. The key issue was not only trading risk, but a failure of supervision, segregation of duties and internal controls.
- **Société Générale, 2008.** Jérôme Kerviel built large unauthorized positions that were concealed from the bank's control system. When the positions were unwound, the loss was about EUR 4.9 billion.
- **JPMorgan "London Whale", 2012.** A portfolio in the Chief Investment Office generated losses of about USD 6.2 billion. Supervisory findings emphasized weaknesses in valuation controls, risk limits and escalation procedures.
- These examples show that operational risk is not only about small processing errors. Weak controls can transform local failures into **extreme losses**.

- **Basel II** introduced operational risk as a separate **Pillar I** capital requirement and allowed banks, under the Advanced Measurement Approach (**AMA**), to use internally developed models subject to supervisory approval.
 - ⇒ The regulatory capital measure had to correspond to the **99.9% VaR of the one-year aggregate operational-loss distribution**, so the modelling problem was essentially to estimate the extreme tail of that distribution accurately.
- **Basel III** removed the AMA from Pillar I. Under the current EU framework, operational-risk capital is determined through a **standardised Pillar I** approach based on the Business Indicator Component, while bank-specific risk is addressed under Pillar II through ICAAP and supervisory review.
 - ⇒ Internal modelling remains relevant within **ICAAP/Pillar II**, where loss data, scenario analysis and stress testing can be used to assess internal capital adequacy, and supervisors may require additional capital if bank-specific risks are not adequately covered.

Why is operational risk difficult to model?

- Internal **datasets are limited**. Losses are sparse over time, and the observations that matter most for tail-risk assessment are precisely the few extreme ones.
- Severity distributions can be extremely **heavy-tailed**. In some applications, estimated tail indices are close to one (Di Clemente and Romano, 2004), so very large losses have a strong impact on high quantiles and expected shortfall.
- Operational risk is **heterogeneous**: fraud, cyber events, system failures, legal losses and execution errors do not follow the same mechanism.
- **Dependence** may appear across business lines and event types when a common episode affects several parts of the institution.

- Operational risk data are recorded at the level of individual loss events. For each event, regulation requires information on gross losses, recoveries, grouped losses and several reference dates, including occurrence, discovery and accounting dates.
- Under Basel II, operational losses were organised through an 8×7 classification based on:
 - **business lines**, describing where in the bank the loss occurred;
 - **event types**, describing the source or nature of the loss.

Thus, each loss could be associated with a Business Line–Event Type risk cell.

- Under the current EU implementation of Basel III (CRR3), operational-loss reporting is moving toward a more granular taxonomy. The EBA draft RTS specify **7 “Level 1” event types**, **26 “Level 2” categories** and **15 attributes**, with each loss event assigned to the applicable categories and attributes.
⇒ Operational-risk data therefore have an increasingly **high-dimensional** and **sparse structure**.

The UniCredit dataset

- We use a dataset of operational losses from **UniCredit**.
 - ⇒ The dataset contains 40,871 operational loss events observed from January 2005 to June 2014.
 - ⇒ The original observations are individual loss events, with a date, an amount and a Basel II event-type classification.
- In this application, losses are classified into $d = 7$ event types:
 - **IFRAUD**: Losses resulting from fraudulent acts committed by employees or other internal parties;
 - **EFRAUD**: Losses resulting from fraudulent acts committed by external parties;
 - **EPWS**: Employment disputes, discrimination, and workplace health and safety issues;
 - **CPBP**: Inappropriate business practices, mis-selling, product issues, or breaches of client obligations;
 - **DPA**: Losses resulting from damage to or destruction of buildings, equipment, or other physical assets;
 - **BDSF**: Events caused by system outages, IT failures, or disruptions affecting business operations;
 - **EDPM**: Events resulting from operational errors, failed transactions, or process execution issues.

Daily data: summary by event type

- The daily data show strong heterogeneity across event types in both frequency and severity.
- **Zero inflation** is substantial for several event types, while a few categories are active on many days.

Event type	# events	% zero days	Total loss	Max daily loss
IFRAUD	1,271	81.9%	193M	31.6M
EFRAUD	6,391	34.9%	140.2M	38M
EPWS	2,292	74.1%	89.6M	15M
CPBP	16,138	32.2%	1251.1M	138.3M
DPA	896	82.7%	9.8M	1.9M
BDSF	674	86.1%	10.6M	0.5M
EDPM	13,209	29.8%	532.4M	54.4M

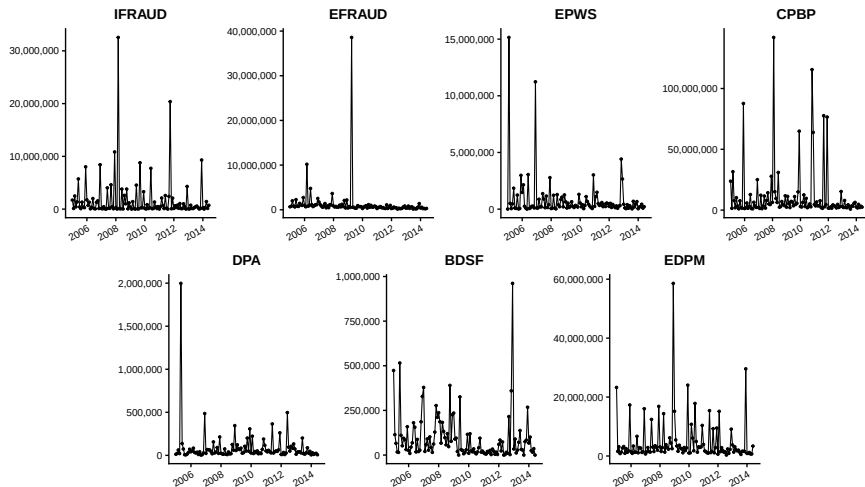
- Let $D_{\tau,j}$ be the daily loss for event type j on day τ .
- Daily losses contain many zeros, because not all event types are active every day.
- We construct **monthly losses** by aggregating daily losses within each event type:

$$X_{t,j} = \sum_{\tau \in \mathcal{M}_t} D_{\tau,j}.$$

- Monthly aggregation reduces zero inflation and gives a first multivariate loss vector:

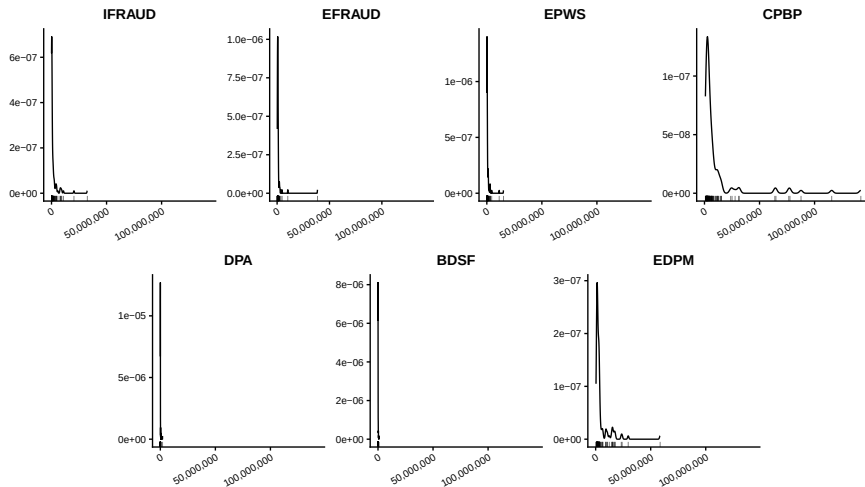
$$X_t = (X_{t,1}, \dots, X_{t,7}).$$

Monthly losses over time



Monthly total losses highlight the irregular and event-driven nature of operational risk.

Marginal densities



Kernel densities of monthly losses by event type.

Heavy-tailed losses

- A loss variable X is heavy-tailed when large losses have non-negligible probability, even far in the right tail.
- A common model for heavy-tailed losses is **regular variation**:

$$\mathbb{P}(X > x) = \bar{F}(x) = x^{-\alpha} L(x), \quad x \rightarrow \infty,$$

where $\alpha > 0$ and $L(x)$ is slowly varying.

- **Slowly varying** means that, for every $c > 0$,

$$\frac{L(cx)}{L(x)} \rightarrow 1, \quad x \rightarrow \infty.$$

- In the far tail, this means that:

$$\bar{F}(x) \approx Cx^{-\alpha}.$$

$\implies$ Smaller values of α correspond to **heavier** tails.

Heavy tails and the tail index

- In extreme value theory, the same tail behavior is often described using the **tail index**:

$$\xi = \frac{1}{\alpha}.$$

⇒ The parameter ξ controls the heaviness of the tail.

⇒ Larger ξ means a heavier tail and more probability assigned to very large losses.

- Moment existence depends on the tail index:

- if $\xi > 1$, the mean is infinite;
- if $\xi < 1$, the mean is finite;
- if $\xi < 1/2$, the variance is finite.

⇒ Heavy tails are important for risk measures and capital estimates.

- A **log-log plot** is a useful visual diagnostic for **regularly varying** tails.
- Suppose that the survival function is approximately Pareto-type in the tail:

$$\bar{F}(x) \approx Cx^{-\alpha}.$$

$\Rightarrow$ Taking logarithms gives:

$$\log \bar{F}(x) \approx \log C - \alpha \log x.$$

- Therefore, if we plot $\log \bar{F}(x)$ against $\log x$, the upper tail should look approximately linear.
 $\Rightarrow$ The **slope** of the line is approximately $-\alpha$.

How to build a log-log tail plot

- 1 Start from positive observations: $X_1, \dots, X_n$.
- 2 Sort them in increasing order: $X_{(1)} \leq \dots \leq X_{(n)}$.

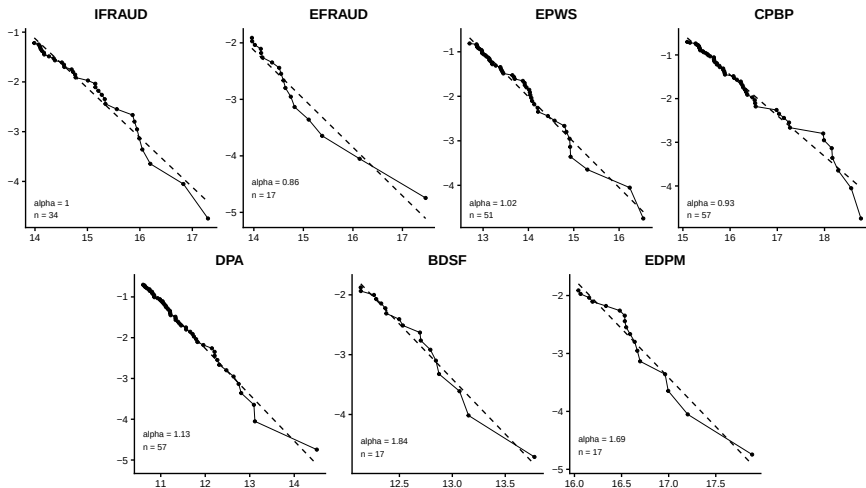
- 3 Estimate the survival probability at each ordered observation:

$$\hat{\bar{F}}(X_{(i)}) = \frac{n - i + 1}{n + 1}.$$

- 4 Then plot the points: $(\log X_{(i)}, \log \hat{\bar{F}}(X_{(i)}))$.

$\Rightarrow$ The diagnostic is mainly about the **upper tail**, not the whole distribution.

Heavy tails across event types



Empirical log-log tail plots provide a first diagnostic of heavy-tailed monthly losses.

From marginal tails to joint tails

- **Marginal** tail diagnostics tell us whether each event type is heavy-tailed.
- For risk management, we also need to understand whether extreme losses occur **together**.
- For two event types j and k , we first transform the marginal distributions to a common scale:

$$U_j = \hat{F}_j(X_j), \quad U_k = \hat{F}_k(X_k).$$

⇒ These are **pseudo-observations** on the uniform scale.

- Equivalently, one may transform to standard Pareto margins
⇒ The tail event is unchanged by monotone marginal transformations.

- Consider two event types with uniform marginal variables U_j and U_k .
- The coefficient of upper tail dependence is:

$$\chi = \lim_{u \rightarrow 1} \mathbb{P}(U_j > u \mid U_k > u).$$

$\implies$ If $\chi > 0$, the variables are **asymptotically dependent**.

$\implies$ If $\chi = 0$, the variables are **asymptotically independent**.

- Asymptotic dependence means that, even at very high levels, an extreme loss in one event type still has a non-negligible probability of being accompanied by an extreme loss in another event type.

The empirical $\chi(u)$ statistic

- In finite samples, we cannot take the limit $u \rightarrow 1$.
- Instead, we compute the diagnostic over a grid of **high thresholds**:

$$u_1, \dots, u_m.$$

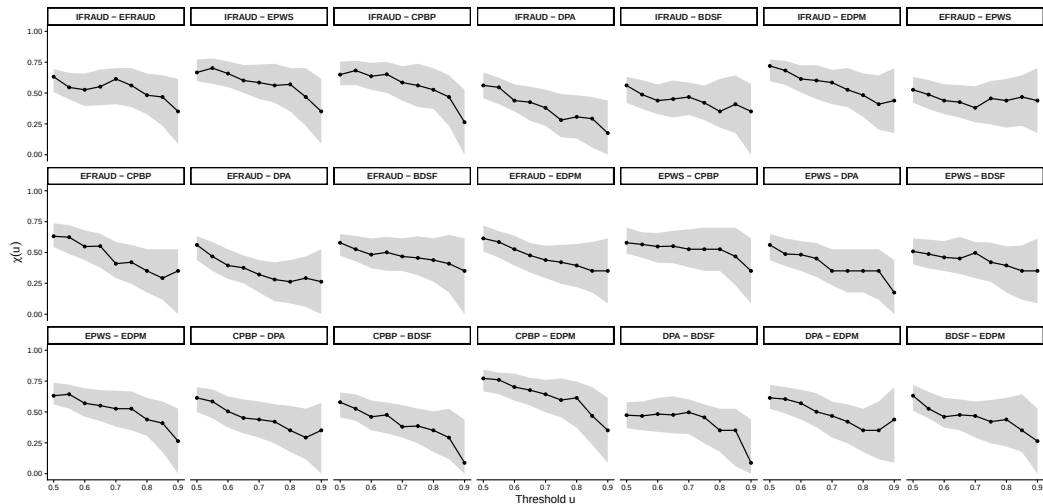
- For a fixed threshold u , the empirical statistic is:

$$\hat{\chi}(u) = \frac{1}{T(1-u)} \sum_{t=1}^T \mathbf{1} \left\{ \hat{U}_{tj} > u, \hat{U}_{tk} > u \right\}.$$

$\Rightarrow$ We use a nonparametric bootstrap to compute confidence intervals

- A curve that stays **away from zero** at high thresholds suggests **stronger** extremal dependence.

Tail dependence diagnostic: $\chi(u)$



Pairwise $\chi(u)$ estimates with pointwise bootstrap confidence bands.

Why do we need $\bar{\chi}$?

- The coefficient $\chi(u)$ separates asymptotic dependence from asymptotic independence. If $\chi > 0$, joint extremes survive in the limit; if $\chi = 0$, the conditional probability of a joint extreme goes to zero.
- However, $\chi = 0$ **does not mean** that all pairs are equally weak in the joint tail. Two pairs can both be asymptotically independent, while one still has much more joint-tail association at high finite levels.
- The statistic $\bar{\chi}$ is used to describe this **residual** tail association. To study it, we first put the margins on a common Pareto-type scale:

$$Z_j = (1 - U_j)^{-1}, \quad Z_k = (1 - U_k)^{-1}.$$

Then $\mathbb{P}(Z_j > z) \sim z^{-1}$ and $\mathbb{P}(Z_k > z) \sim z^{-1}$.

The Ledford–Tawn joint-tail model

- Ledford and Tawn (1996) describe residual tail association after marginal standardization. On the Pareto-type scale, the joint tail is modelled as:

$$\mathbb{P}(Z_j > z, Z_k > z) = L(z)z^{-1/\eta}, \quad z \rightarrow \infty,$$

where $\eta \in (0, 1]$ and $L(z)$ is slowly varying.

- The parameter η controls the decay rate of the joint tail probability. Since $\mathbb{P}(Z_j > z) \sim z^{-1}$ and $\mathbb{P}(Z_k > z) \sim z^{-1}$, independence gives:

$$\mathbb{P}(Z_j > z, Z_k > z) \sim z^{-2}.$$

- In the Ledford–Tawn model, the joint tail behaves like $z^{-1/\eta}$, up to a slowly varying term.
 - If $\eta < 1/2$, then the joint tail decays faster than under independence.
 - If $\eta = 1/2$, then the joint tail has the same rate as independence.
 - If $\eta > 1/2$, then the joint tail decays more slowly than under independence.
 - If $\eta = 1$, then the joint tail has the same order as a marginal tail, compatible with asymptotic dependence.

- Ledford and Tawn summarize residual tail association through:

$$\bar{\chi} = 2\eta - 1.$$

- This rescaling gives a simple interpretation:
 - $\eta < 1/2 \implies \bar{\chi} < 0$: weaker joint-tail association than independence;
 - $\eta = 1/2 \implies \bar{\chi} = 0$: independence benchmark;
 - $1/2 < \eta < 1 \implies 0 < \bar{\chi} < 1$: positive residual tail association;
 - $\eta = 1 \implies \bar{\chi} = 1$: limiting case compatible with asymptotic dependence.
- Values of $\bar{\chi}$ closer to one indicate stronger residual extremal association, while values close to zero are consistent with behavior close to independence in the joint tail.

Empirical $\bar{\chi}(u)$

- Let $s = 1 - u$ and let $p_{jk}(u) = \mathbb{P}(U_j > u, U_k > u)$ be the joint exceedance probability. Since $U_j > u$ is equivalent to $Z_j > 1/(1 - u) = 1/s$, the Ledford–Tawn model gives: $p_{jk}(u) \approx L(1/s)s^{1/\eta}$. Ignoring the slowly varying term in a finite-threshold diagnostic gives:

$$\bar{\chi}(u) = \frac{2 \log(1 - u)}{\log p_{jk}(u)} - 1.$$

- In practice, we use pseudo-observations $\hat{U}_{t,j} = \hat{F}_j(X_{t,j})$ and $\hat{U}_{t,k} = \hat{F}_k(X_{t,k})$, and estimate:

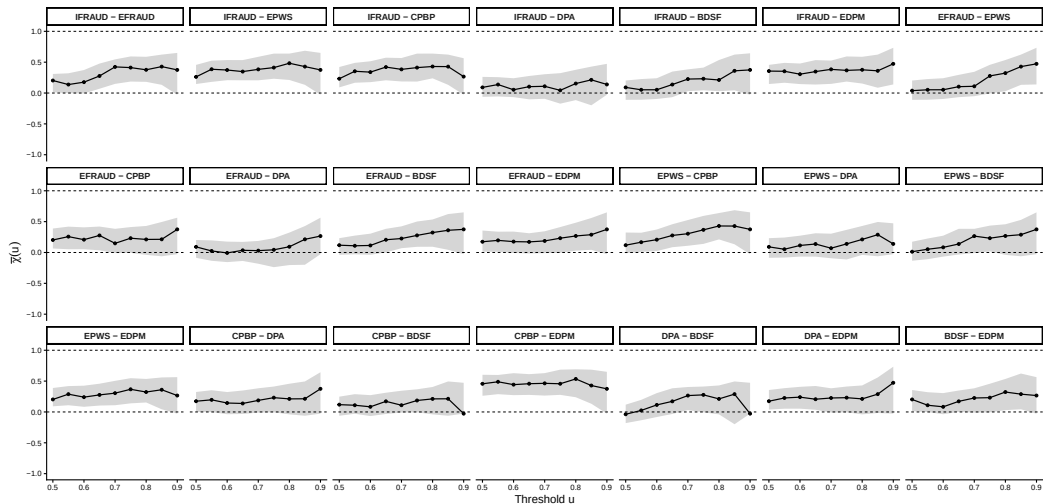
$$\hat{p}_{jk}(u) = \frac{1}{T} \sum_{t=1}^T \mathbf{1} \{ \hat{U}_{t,j} > u, \hat{U}_{t,k} > u \}.$$

Then compute:

$$\hat{\bar{\chi}}(u) = \frac{2 \log(1 - u)}{\log \hat{p}_{jk}(u)} - 1.$$

$\implies$ The statistic is evaluated over the same grid of high thresholds used for $\chi(u)$.

Tail dependence diagnostic: $\bar{\chi}(u)$



Pairwise $\bar{\chi}(u)$ estimates with pointwise bootstrap confidence bands.

Synthetic operational loss generation

Limits of internal operational loss data

- Internal operational-loss data are limited exactly where risk measurement is most difficult: severe losses are rare, so a single institution may observe only a few extreme events over many years.
- Basel II already required AMA models to **combine internal loss** data with external data, scenario analysis and business environment/internal control factors.
- Although internal models no longer determine Pillar I capital, the same need remains under ICAAP/Pillar II: banks must assess their own operational-risk profile using information that goes beyond the short internal loss history.
- The observed dataset should therefore be viewed as a **partial sample** of the operational-risk universe, rather than a complete representation of the losses the institution may face.

Why synthetic operational loss data?

- Synthetic data provide a way to generate **alternative loss histories** that are plausible under the statistical structure learned from the observed data.
 - ⇒ By generating many alternative histories, one can explore loss configurations that are not directly observed in the internal dataset.
- Operational-loss data are also highly **sensitive**: they may reveal information on fraud, litigation, cyber incidents, process failures or weaknesses in internal controls, which makes data sharing across institutions difficult.
 - ⇒ Synthetic datasets may facilitate data sharing, benchmarking and methodological research without directly releasing the original loss records.
- External operational-loss data are often obtained from commercial databases or industry consortia and may be **costly**, incomplete or difficult to access.
 - ⇒ A sufficiently accurate generative model could reduce the dependence on repeatedly accessing large external datasets for simulation and model development.

Synthetic financial data and the operational risk gap

- In finance, synthetic data methods have mostly been developed for **market** and **credit** applications (Potluru et al., 2023).
 - ⇒ These data often have a clearer observation structure: market variables are observed frequently, credit data are usually organized around borrowers or portfolios
- Operational risk data are **different**: losses are event-driven, irregularly observed, classified by event type and often zero-inflated at daily frequency, with many days showing no recorded loss and a few days containing large positive losses.
- Therefore, synthetic operational risk data are not just another financial time-series problem. They require a generative model that can handle sparsity, heterogeneity, heavy tails and dependence across event types.

What should synthetic losses preserve?

- For operational risk, a synthetic dataset is useful only if it preserves the features that drive risk measurement, not only the average behavior of the data.
 - At the **marginal level**, synthetic losses should reproduce the distribution of each event type, especially tail thickness and the relative importance of very large losses.
 - At the **dependence level**, synthetic losses should reproduce co-movements across event types, including the probability that large losses occur together in the same period.
 - At the **aggregate level**, the synthetic dataset should reproduce the distribution of total losses, for example monthly or annual sums across event types:

$$S_t = \sum_{j=1}^d X_{t,j}.$$

⇒ This also means preserving risk measures (VaR/ES) computed from the aggregate loss distribution.

Tail-aware GAN methods

Learning a data-generating distribution

- We observe an empirical sample of multivariate observations

$$X_1, \dots, X_T \stackrel{\text{i.i.d.}}{\sim} P_{\text{data}}, \quad X_t = (X_{t,1}, \dots, X_{t,d}) \in \mathcal{X} \subseteq \mathbb{R}^d,$$

where T is the sample size, d is the dimension of one observation, and P_{data} is the **unknown data-generating distribution (DGD)**.

- The goal is to **learn** a model distribution that is close enough to P_{data} to generate new observations

$$X_1^*, \dots, X_m^*, \quad X_t^* = (X_{t,1}^*, \dots, X_{t,d}^*),$$

whose distribution is close to the DGD.

- For example, if one observation is a vector of seven monthly losses, then

$$X_t = (X_{t,1}, \dots, X_{t,7}),$$

where $X_{t,j}$ is the loss of event type j in month t .

The GAN construction

- A **Generative Adversarial Network (GAN)** constructs a model distribution P_θ approximating the data-generating distribution P_{data} through the interaction of two **neural networks**:
 - The **generator**, G_θ , transforms random latent inputs into synthetic observations.
 - The **discriminator**, D_ϕ , solves an auxiliary classification problem in which it tries to distinguish real observations from generated ones.

⇒ The vectors θ and ϕ contain the trainable parameters of the two networks.
- Training is adversarial because the two networks have **opposite objectives**:
 - D_ϕ tries to detect generated observations;
 - G_θ tries to make them undetectable.

⇒ If training succeeds, the distribution P_θ induced by G_θ becomes **close** to P_{data} .

The generator

- The **generator** is a parametric transformation, $G_\theta : \mathbb{R}^k \rightarrow \mathbb{R}^d$, where k is the dimension of the latent space and d is the dimension of one observation in the original data space.
- The latent dimension k is selected by the modeller and is usually treated as a **hyperparameter** to be investigated together with network depth, width, regularization and optimization settings.
- A latent random vector, $Z_t = (Z_{t,1}, \dots, Z_{t,k})' \sim P_Z$ is drawn from a simple reference distribution, e.g. multivariate Gaussian. The generator maps this vector into one synthetic observation:

$$X_t^* = G_\theta(Z_t) \in \mathbb{R}^d.$$

- Repeating the procedure with independent draws $Z_1, \dots, Z_m$ produces a synthetic sample:

$$X_t^* = G_\theta(Z_t), \quad t = 1, \dots, m.$$

The discriminator

- The **discriminator** is a function $D_\phi : \mathbb{R}^d \rightarrow (0, 1)$. Conceptually, it processes one observation $x \in \mathbb{R}^d$ at a time and returns one classification probability $D_\phi(x)$. Introduce a binary class indicator

$$C = \begin{cases} 1, & \text{if the observation is drawn from } P_{\text{data}}, \\ 0, & \text{if the observation is produced by } G_\theta. \end{cases}$$

$\Rightarrow$ The discriminator models the conditional probability $\mathbb{P}(C = 1 \mid X)$.

- During training, many observations are processed simultaneously for computational efficiency. Given b real observations and b generated observations, the discriminator returns

$$D_\phi(X_1), \dots, D_\phi(X_b), D_\phi(X_1^*), \dots, D_\phi(X_b^*).$$

The GAN objective

- The classical GAN is defined by the **minimax problem**:

$$\min_{\theta} \max_{\phi} V(\phi, \theta)$$

with

$$V(\phi, \theta) = \mathbb{E}_{X \sim P_{\text{data}}} [\log D_{\phi}(X)] + \mathbb{E}_{Z \sim P_Z} [\log(1 - D_{\phi}(G_{\theta}(Z)))] .$$

- $\implies$ For a fixed generator G_{θ} , the discriminator maximizes a binary classification objective.
 - $\implies$ For a fixed discriminator D_{ϕ} , the generator modifies $G_{\theta}(Z)$ so that generated observations become harder to distinguish from real ones.
- If training succeeds, the generated distribution P_{θ} **moves closer** to the data distribution P_{data} .

Why vanilla GANs struggle with heavy tails

- Heavy-tailed data create three distinct **challenges** for standard GANs.
 - **Data scale.** Heavy-tailed marginals can span very different orders of magnitude, while only a small fraction of the sample lies in the far tail. This can make training numerically unbalanced and make extreme observations difficult to learn directly on the original scale.
 - **Generator representation.** The tail behaviour of the generated data is constrained by both the latent distribution and the generator architecture. With bounded latent noise, standard ReLU-type generators produce bounded outputs; with Gaussian latent noise, they typically produce light-tailed outputs. Hence, a vanilla generator may be structurally unable to represent Pareto-type tails.
 - **Loss function.** Even if the generator can represent heavy tails, standard training objectives may become unstable or even ill-defined when extreme observations dominate distances or when the relevant moments do not exist.

From standard GANs to tail-aware generative models

- **Tail-aware approaches** modify one or more components of the standard GAN pipeline in order to introduce stronger structure or a stronger learning signal in extreme regions.
 - A first strategy is to **transform the data before training**. Heavy-tailed marginals are modelled separately, the data are mapped to a more regular scale, the GAN is trained mainly to learn dependence, and the generated data are transformed back.
⇒ This motivates **pre-transformation** approaches (Wiese et al., 2020; Boulaguiem et al., 2022).
 - A second strategy is to **modify the generator architecture**. The idea is to build extreme-value structure into the generator, so that heavy-tailed quantile behavior can be represented more directly.
⇒ This motivates **EV-GAN** type approaches (Allouche et al., 2022).
 - A third strategy is to **modify the latent space and the training metric**. Heavy-tailed latent inputs and losses adapted to heavy-tailed distributions can make tail generation more stable and meaningful.
⇒ This motivates **Pareto GAN** and related heavy-tail-based architectures (Huster et al., 2021).

Method 1: Vanilla GAN with marginal transformations

Motivation and starting point

- We observe T multivariate loss vectors $X_t = (X_{t,1}, \dots, X_{t,d})$, $t = 1, \dots, T$, where the component $X_{t,j}$ is the loss associated with event type j during period t . Each component has a different marginal cumulative distribution function:

$$F_j(x) = \mathbb{P}(X_{t,j} \leq x), \quad j = 1, \dots, d.$$

- A Vanilla GAN trained on the original loss scale must learn both the **marginal** tails and the **dependence** across event types.

$\implies$ With few extreme observations, learning both components simultaneously may be **difficult**.

- We therefore separate two statistical tasks:
 - **Marginal modelling**: the marginal distributions are estimated before training;
 - **Dependence learning**: the GAN mainly learns the dependence structure.

Separating marginals and dependence

- For continuous marginal distributions, Sklar's theorem gives:

$$\mathbb{P}(X_{t,1} \leq x_1, \dots, X_{t,d} \leq x_d) = C(F_1(x_1), \dots, F_d(x_d)),$$

where the function $C : [0, 1]^d \rightarrow [0, 1]$ is the **copula** of the multivariate distribution.

- For each event type j , we estimate $\hat{F}_j$ and transform each loss into a **pseudo-uniform** observation:

$$U_{t,j} = \hat{F}_j(X_{t,j}), \quad U_t = (U_{t,1}, \dots, U_{t,d}).$$

If $\hat{F}_j$ is a good approximation of F_j , then $U_{t,j} \approx U(0, 1)$.

$\implies$ Marginal scales are removed, while the dependence information is approximately preserved.

- The transformation depends on how we estimate each marginal distribution $\hat{F}_j$. We consider:
 - a fully **empirical distribution**;
 - an empirical body combined with a **Generalized Pareto tail**;
 - a parametric Tukey ***g-and-h* distribution**.

Alternative models for the marginal distributions

- The **empirical distribution**: $\hat{F}_{j,\text{emp}}(x) = \frac{1}{T} \sum_{t=1}^T \mathbb{1}\{X_{t,j} \leq x\}$.
 $\implies$ fully data-driven, but it **cannot extrapolate** beyond the observed sample maximum.
- An empirical body can be combined with a **Generalized Pareto tail** above a high threshold τ_j :

$$\hat{F}_j(x) = \begin{cases} \hat{F}_{j,\text{emp}}(x), & x \leq \tau_j, \\ \hat{p}_j + (1 - \hat{p}_j) G_{\hat{\xi}_j, \hat{\beta}_j}(x - \tau_j), & x > \tau_j. \end{cases}$$

$\implies$ empirical flexibility in the body, EVT-based extrapolation in the tail.

- The **Tukey g -and- h distribution** provides a parametric model for the **whole** marginal distribution:

$$X = a + b \frac{\exp(gW) - 1}{g} \exp\left(\frac{hW^2}{2}\right), \quad W \sim N(0, 1),$$

where a and b control location and scale, g asymmetry, and h tail heaviness.

A common target space

- After the probability integral transform, every component is approximately uniform.
 $\implies$ We may train the GAN directly on U_t , or map the uniforms to a common target distribution.

- Let H be the cumulative distribution function of the target distribution. We define:

$$Y_{t,j} = H^{-1}(U_{t,j}).$$

Examples include:

- uniform margins: $H^{-1}(u) = u$;
 - Gaussian margins: $H^{-1}(u) = \Phi^{-1}(u)$;
 - standard Gumbel margins: $H^{-1}(u) = -\log[-\log(u)]$.
- The target distribution **matters**: uniform margins compress extremes near the boundaries, while Gaussian or Gumbel targets spread tail observations over a wider scale (Peard et al. 2026).

- The training transformation is applied **component-wise** to $X_t = (X_{t,1}, \dots, X_{t,d})$:

$$X_{t,j} \longrightarrow U_{t,j} = \hat{F}_j(X_{t,j}) \longrightarrow Y_{t,j} = H^{-1}(U_{t,j}), \quad t = 1, \dots, T, \quad j = 1, \dots, d.$$

- A GAN is trained on the **transformed vectors** $Y_1, \dots, Y_T$. After training, for one synthetic observation:
 - 1 latent noise $Z^* \sim P_Z$ is mapped into a synthetic transformed vector $Y^* = G_\theta(Z^*)$;
 - 2 we return to the uniform scale component by component $U_j^* = H(Y_j^*)$;
 - 3 finally, synthetic losses are obtained through: $X_j^* = \hat{F}_j^{-1}(U_j^*)$, with $j = 1, \dots, d$.
- Repeating the procedure produces a synthetic sample $X_1^*, \dots, X_m^*$, where each $X_t^* = (X_{t,1}^*, \dots, X_{t,d}^*)$.

Method 2: EV-GAN

Why can a classical ReLU generator fail?

- A generator learns a transformation G such that $G(Z) \stackrel{d}{\approx} X$, where $X \sim P_{\text{data}}$. In the univariate case, if $Z \sim U(0, 1)$, the exact latent-to-data map is the quantile function:

$$X \stackrel{d}{=} q_X(Z), \quad q_X(u) = F_X^{-1}(u).$$

$\implies$ With uniform latent noise, the ideal generator can be viewed as $G(u) = q_X(u)$.

- If P_{data} is **unbounded above**, then $q_X(u) \rightarrow +\infty$ as $u \rightarrow 1$. For a Pareto-type distribution, the issue is more severe because extreme quantiles diverge at a **power rate**.
- Consider a **ReLU neural network**: $G_\theta(z) = \sum_{r=1}^J a_r \sigma(w_r z + b_r)$, with $\sigma(x) = \max(x, 0)$. If Z has bounded support, then $G_\theta(Z)$ is **bounded**; therefore $Z \sim U(0, 1)$ cannot be mapped by a finite ReLU network into an unbounded distribution.
- **EV-GAN** (Allouche et al., 2022) changes the object that the neural network is asked to approximate.

The Tail-Index Function

- EV-GAN considers Pareto-type distributions, whose survival function satisfies:

$$\bar{F}_X(x) = x^{-1/\xi} \ell_X(x), \quad x \rightarrow \infty.$$

- Instead of approximating the diverging quantile $q_X(u)$ directly, define:

$$H_u(y) = \frac{-\log y}{\log(1-u^2) - \log 2}, \quad f^{\text{TIF}}(u) = H_u(q_X(u)).$$

- For a Pareto-type tail, $\log q_X(u) \sim -\xi \log(1-u)$ and $\log((1-u^2)/2) \sim \log(1-u)$, so:

$$f^{\text{TIF}}(u) \rightarrow \xi \quad \text{as } u \rightarrow 1.$$

- Hence the transformation converts the **learning objective** of the generator:

$$q_X(u) \rightarrow +\infty \quad \implies \quad f^{\text{TIF}}(u) \rightarrow \xi.$$

$\implies$ The NN now approximates a bounded continuous function, which is much easier for a ReLU to learn.

From TIF to CTIF

- Although bounded, f^{TIF} may still have irregular derivatives near $u = 1$.
- EV-GAN therefore introduces a corrected function:

$$f^{\text{CTIF}}(u) = f^{\text{TIF}}(u) - \sum_{\ell=1}^6 \kappa_{\ell} e_{\ell}(u),$$

where $e_1, \dots, e_6$ are fixed correction functions and $\kappa_1, \dots, \kappa_6$ are distribution-specific coefficients.

- The neural network approximates the smoother object:

$$\hat{f}^{\text{CTIF}}(u) = \sum_{r=1}^J a_r \sigma(w_r u + b_r),$$

and the TIF is reconstructed as:

$$\hat{f}^{\text{TIF}}(u) = \hat{f}^{\text{CTIF}}(u) + \sum_{\ell=1}^6 \kappa_{\ell} e_{\ell}(u).$$

The EV-GAN generator

- The inverse transformation:

$$H_z^{-1}(x) = \left(\frac{1 - z^2}{2} \right)^{-x}$$

implicitly reconstructs the heavy-tailed quantile map. The EV-GAN generator is therefore:

$$G_{\psi}^{\text{TIF}}(z) = H_z^{-1} \left(\sum_{r=1}^J a_r \sigma(w_r z + b_r) + \sum_{\ell=1}^6 \kappa_{\ell} e_{\ell}(z) \right).$$

- Conceptually:

$$z \rightarrow \hat{f}^{\text{CTIF}}(z) \rightarrow \hat{f}^{\text{TIF}}(z) \rightarrow \hat{q}_X(z) \rightarrow \tilde{X}.$$

$\implies$ The neural network learns the regular part of the latent-to-data map;

$\implies$ The inverse EVT transformation restores the unbounded heavy-tail behavior.

Extending EV-GAN to the multivariate case

- Let $X = (X_1, \dots, X_d)$ be a generic heavy-tailed random vector and let $Z = (Z_1, \dots, Z_k) \sim U([0, 1]^k)$.
- EV-GAN extends the univariate construction coordinate-wise: for marginal j ,

$$G_{\psi}^{\text{TIF}^j}(Z) = H_{Z_j}^{-1} \left(\hat{f}^{\text{TIF}^j}(Z) \right), \quad j = 1, \dots, d,$$

which **requires** $k \geq d$.

$\implies H_{Z_j}^{-1}$ uses only the coordinate Z_j and controls the heavy-tailed scale of marginal j .

$\implies$ The neural-network term $\hat{f}^{\text{TIF}^j}(Z)$ can depend on the full latent vector $Z = (Z_1, \dots, Z_k)$.

Method 3: Pareto GAN

Why modify the latent distribution?

- A standard GAN usually starts from a simple latent distribution, such as uniform or Gaussian noise, and relies on the generator to transform this noise into realistic observations.
- For heavy-tailed data, this can be **problematic**.
 - ⇒ If the latent input is bounded, a finite ReLU (piecewise linear) generator produces bounded outputs.
 - ⇒ If the latent input is light-tailed, such as Gaussian noise, a finite ReLU (piecewise-linear) generator produces light-tailed outputs.
- This suggests a different strategy: instead of asking the neural network to create a power-law tail from light-tailed noise, give the generator **heavy-tailed latent inputs** from the start.
- This is the starting point of **Pareto GAN** (Huster et al., 2021): extreme values are already present before training, and the generator learns how to transform and combine them.

A heavy-tailed latent space

- A convenient EVT-based choice is a **Generalized Pareto** variable:

$$Z_\xi = \frac{U^{-\xi} - 1}{\xi}, \quad U \sim U(0, 1),$$

with

$$\mathbb{P}(Z_\xi > z) = (1 + \xi z)^{-1/\xi}.$$

- $\Rightarrow$ The GPD is used as **latent noise**: it supplies a controlled Pareto-type asymptotic tail, while the neural network remains free to learn the body and the dependence structure of the data.
- $\Rightarrow$ Because a finite ReLU neural network is piecewise linear, if $G_\theta(Z_\xi)$ is unbounded in a given direction, its marginal tail **preserves** the latent extreme-value index ξ .
- $\Rightarrow$ The latent distribution determines the asymptotic tail class available to the generator, so the network does not need to “manufacture” a power-law tail from light-tailed noise.

Pareto GAN: marginal tail indices

- We start from the observed sample $X_t = (X_{t,1}, \dots, X_{t,d})$, $t = 1, \dots, T$, and estimate one marginal tail index for each component, $\hat{\xi} = (\hat{\xi}_1, \dots, \hat{\xi}_d)$. These estimates summarize the asymptotic tail heaviness of the observed marginals.
- The generator starts from a k -dimensional latent vector $Z = (Z_1, \dots, Z_k)$, whose components are GPD random variables with common tail index $\xi = 1$. This common index is only a reference scale: different marginal tail indices are introduced after the neural-network mapping.
- The neural network first produces an intermediate synthetic vector $Y^* = G_\theta(Z) = (Y_1^*, \dots, Y_d^*)$. Since the unbounded coordinates of Y^* inherit the latent GPD tail index 1, a coordinate-specific power transformation is applied to the output of the neural network:

$$X_j^* = \text{sign}(Y_j^*) |Y_j^*|^{\hat{\xi}_j}, \quad j = 1, \dots, d.$$

Pareto GAN: loss function

- The classical GAN loss is an **adversarial classification** loss. This loss could be used with a heavy-tailed generator, but it does not directly compare the tail behavior of the data.
- Huster et al. (2021) train the generator with a distance-based loss that directly compares the data distribution and the generated distribution, such as Wasserstein distance or energy distance.
- In the experiments, the paper uses the **energy distance**. Let $X, X' \sim P_{\text{data}}$ and $X^*, X^{*'} \sim P_{\theta}$ be independent. For a metric d ,

$$\mathcal{E}_d(P_{\text{data}}, P_{\theta}) = 2\mathbb{E}[d(X, X^*)] - \mathbb{E}[d(X, X')] - \mathbb{E}[d(X^*, X^{*'})].$$

The first term compares real and generated observations, while the last two terms correct for the internal variability of each distribution. The generator is trained by solving:

$$\min_{\theta} \mathcal{E}_d(P_{\text{data}}, P_{\theta}), \quad X^* = G_{\theta}(Z).$$

⇒ Compared with the classical minimax objective, the parameters of the generator are estimated by **minimizing** a statistical distance between real and generated distributions.

Pareto GAN: loss function

- Since the loss contains expectations of distances, the metric d becomes crucial. With the usual Euclidean metric, the loss contains expectations of distances such as $d(X, X^*) = \|X - X^*\|_2$. For heavy-tailed distributions, these expectations may be **infinite**; even when computed empirically, mini-batch estimates can be dominated by a few extreme observations.

- Pareto GAN replaces the Euclidean metric with the **root-Euclidean metric**:

$$d_\gamma(x, y) = \|x - y\|_2^{1/\gamma}, \quad \gamma \geq 1.$$

When $\gamma > 1$, large distances are compressed but remain ordered and distinguishable.

- For a GPD with tail index ξ , the first moment on the metric space $(\mathbb{R}, d_\gamma)$ is **finite** whenever $\gamma > \xi$. Therefore, choosing γ larger than the relevant tail indices makes the energy distance finite while keeping useful tail information in the training loss.

Pareto GAN: the complete multivariate model

- **Tail estimation:** estimate the marginal extreme-value indices $\hat{\xi}_1, \dots, \hat{\xi}_d$ from the observed data.
- **Generation:** draw a k -dimensional latent vector with base GPD tail index 1, generate $Y^* = G_\theta(Z)$, and transform each output coordinate as

$$X_j^* = \text{sign}(Y_j^*) |Y_j^*|^{\hat{\xi}_j}, \quad j = 1, \dots, d.$$

Thus the network learns the multivariate distribution while the marginal asymptotic tail indices are imposed by construction.

- **Training:** estimate θ by minimizing energy distance computed with $d_\gamma(x, y) = \|x - y\|_2^{1/\gamma}$. In the multivariate experiment, the paper chooses

$$\gamma = 1 + \max_{j=1, \dots, d} \hat{\xi}_j,$$

a simple conservative choice that ensures finiteness for all estimated marginal tail indices.

Empirical results

Empirical application

- We return to the **UniCredit** operational loss dataset introduced earlier.
- The empirical sample contains:
 - $T = 114$ monthly observations
 - $d = 7$ event types
- Each model is estimated using the same observed sample. After training, we **generate**:

$$m = 10,000$$

synthetic monthly loss vectors for model evaluation.

- We **assess** whether the generated data reproduce:
 - the marginal upper tails;
 - the dependence across event types;
 - the distribution of aggregate monthly losses.

Evaluation criteria

- **Marginal tails: Mean Squared Logarithmic Error (MSLE).**

$$\text{MSLE}(p_0) = \frac{1}{dK_{p_0}} \sum_{j=1}^d \sum_{p_\ell \geq p_0} \left[\frac{\log \hat{q}_j^{\text{gen}}(p_\ell) - \log \hat{q}_j^{\text{emp}}(p_\ell)}{\log 2} \right]^2.$$

quantile $q(p_\ell)$ at the upper-tail probability levels p_ℓ , j indexes event types, and K_ξ the number of levels above ξ . We consider $p_0 \in \{0.90, 0.95\}$. **Lower** values are better.

- **Dependence: Absolute Kendall Error (AKE).**

$$\text{AKE} = \frac{2}{d(d-1)} \sum_{j < k} W_1 \left(\hat{K}_{jk}^{\text{emp}}, \hat{K}_{jk}^{\text{gen}} \right).$$

(j, k) indexes pairs of event types, $\hat{K}_{jk}$ is the empirical Kendall distribution, and W_1 is the Wasserstein-1 distance between observed and generated Kendall distributions. **Lower** values are better.

- **Aggregate risk: Signed Relative Discrepancy (SRD).**

$$S_t = \sum_{j=1}^d X_{t,j}, \quad \text{SRD}_\alpha = \frac{\widehat{\text{VaR}}_\alpha^{\text{gen}} - \widehat{\text{VaR}}_\alpha^{\text{emp}}}{\widehat{\text{VaR}}_\alpha^{\text{emp}}}.$$

S_t is the aggregate monthly loss. We consider $\alpha = \{0.90, 0.95\}$. Values **close to zero** are better.

- **Preprocessing.**

Raw losses, median scaling, and marginal transformations.

- **Target distribution for transformed GANs.**

Gaussian, Uniform, and Gumbel targets, keeping the latent distribution fixed.

- **Marginal modelling.**

Empirical CDF, empirical body with GPD tail, and Tukey g -and- h .

- **Extreme-value GAN architectures.**

EV-GAN and Pareto GAN.

GANs: preprocessing

We compare training on the **original** loss scale, **median scaling**, and **pseudo-uniform** transformed data obtained with the empirical marginal distribution, using Gaussian latent noise.

Marginal tails: MSLE

Preprocessing	0.90	0.95
Raw data	1085.172	1085.860
Median scaled	7.345	10.318
Pseudo-uniform	3.910	5.086

Dependence: AKE

Preprocessing	AKE
Raw data	0.248
Median scaled	0.106
Pseudo-uniform	0.094

Aggregate VaR: SRD

Preprocessing	0.90	0.95
Raw data	-1.000	-1.000
Median scaled	-0.576	-0.753
Pseudo-uniform	-0.695	-0.799

GANs with transformed marginals: target distribution

We keep the empirical marginal distribution transformation and Gaussian latent noise fixed, and compare **Uniform**, **Gaussian** and **Gumbel** target distributions.

Marginal tails: MSLE

Target	0.90	0.95
Gaussian	4.249	5.737
Uniform	3.910	5.086
Gumbel	1.485	2.067

Dependence: AKE

Target	AKE
Gaussian	0.106
Uniform	0.094
Gumbel	0.083

Aggregate VaR: SRD

Target	0.90	0.95
Gaussian	-0.363	-0.472
Uniform	-0.695	-0.799
Gumbel	-0.391	-0.620

GANs with transformed marginals: marginal specification

We keep the Gumbel target and Gaussian latent distribution fixed and compare alternative models for the **margins**.

Marginal tails: MSLE

Marginal model	0.90	0.95
Empirical	1.485	2.067
Empirical+GPD 0.85	1.129	1.491
Empirical+GPD 0.90	1.367	1.842
Empirical+GPD 0.95	1.497	2.023
Tukey g -and- h	2.420	3.355

Dependence: AKE

Marginal model	AKE
Empirical	0.0833
GPD 0.85	0.0803
GPD 0.90	0.0802
GPD 0.95	0.0826
Tukey g -and- h	0.0829

Aggregate VaR: SRD

Marginal model	0.90	0.95
Empirical	-0.391	-0.620
Empirical+GPD 0.85	-0.339	-0.563
Empirical+GPD 0.90	-0.354	-0.575
Empirical+GPD 0.95	-0.375	-0.587
Tukey g -and- h	-0.377	-0.576

Transformed GANs vs extreme-value GAN architectures

We compare the **EV-GAN** and **Pareto GAN** with the **transformed** specification based on Empirical+GPD 0.85 and Gumbel target.

Marginal tails: MSLE

Model	0.90	0.95
Transformed Margins	1.129	1.491
EV-GAN	2.355	3.749
Pareto GAN	2.154	4.238

Dependence: AKE

Model	AKE
Transformed Margins	0.0803
EV-GAN	0.0446
Pareto GAN	0.0232

Aggregate VaR: SRD

Model	0.90	0.95
Transformed Margins	-0.339	-0.563
EV-GAN	-0.483	-0.656
Pareto GAN	-0.017	-0.066

Empirical results: main findings

- Training a Vanilla GAN directly on raw operational losses is ineffective.
⇒ **Preprocessing** and marginal transformation are essential.
- Within transformed GANs, the **Gumbel** target provides the strongest marginal-tail and dependence fit.
- Combining the empirical distribution with a **GPD tail** further improves marginal-tail reproduction, while dependence changes comparatively little.
- Extreme-value GAN architectures perform particularly well in reproducing dependence.
- There is **no best specification**: the preferred model depends on whether the priority is marginal tails, dependence or aggregate risk. Pareto GAN provides the strongest dependence fit and the smallest aggregate VaR discrepancy, whereas transformed GANs provide the strongest marginal-tail fit.

References

- ① Allouche, M., Girard, S., & Gobet, E. (2022). EV-GAN: Simulation of extreme events with ReLU neural networks. *Journal of Machine Learning Research*, 23(150), 1-39.
- ② Boulaguiem, Y., Zscheischler, J., Vignotto, E., van der Wiel, K., & Engelke, S. (2022). Modeling and simulating spatial extremes by combining extreme value theory with generative adversarial networks. *Environmental Data Science*, 1, e5.
- ③ Di Clemente, A., & Romano, C. (2004). A copula-extreme value theory approach for modelling operational risk. *Operational Risk Modelling and Analysis*, 189-208.
- ④ Huster, T., Cohen, J., Lin, Z., Chan, K., Kamhoua, C., Leslie, N. O., ... & Sekar, V. (2021). Pareto GAN: Extending the representational power of gans to heavy-tailed distributions. In *International Conference on Machine Learning* (pp. 4523-4532). PMLR.
- ⑤ Ledford, A. W., & Tawn, J. A. (1996). Statistics for near independence in multivariate extreme values. *Biometrika*, 83(1), 169-187.
- ⑥ Peard, A., Mo, Y., & Hall, J. W. (2026). Simulating spatial multi-hazards with generative deep learning. *Natural Hazards and Earth System Sciences*, 26(4), 1663-1683.
- ⑦ Potluru, V. K., Borrajo, D., Coletta, A., Dalmasso, N., El-Laham, Y., Fons, E., ... & Balch, T. (2023). Synthetic data applications in finance. *arXiv:2401.00081*.
- ⑧ Wiese, M., Knobloch, R., Korn, R., & Kretschmer, P. (2020). Quant GANs: deep generation of financial time series. *Quantitative Finance*, 20(9), 1419-1440.